

Documentación Técnica — Canje FTT

Documento único que consolida toda la documentación técnica del proyecto (arquitectura, instalación, modelo de datos, API, seguridad, tiempo real, pruebas y estado conocido del sistema), verificado contra el código fuente actual del repositorio.

Este archivo reemplaza como referencia principal a [README.md](#), [ARQUITECTURA.md](#), [SETUP.md](#), [CAMBIOS.md](#), [OPTIMIZACIONES_BACKEND.md](#), [CANJE_MASIVO_Y_MEJORAS_UI.md](#), [ACTUALIZACIONES_TIEMPO_REAL.md](#), [CHECKLIST.md](#), [GUIA_RAPIDA.md](#) y [RESUMEN_PROYECTO.md](#), cuyo contenido quedó disperso y, en varios casos, desactualizado (documentaban una versión anterior del proyecto, con otro nombre de evento). Esos archivos se conservan en [EVIDENCIAS_PASANTIA/](#) como referencia histórica, pero no deben tomarse como fuente de verdad del estado actual del sistema — este documento sí.

Índice

1. [Descripción general](#)
2. [Stack tecnológico](#)
3. [Estructura del proyecto](#)
4. [Roles y permisos](#)
5. [Modelo de datos](#)
6. [API — Endpoints](#)
7. [Actualizaciones en tiempo real \(Socket.IO\)](#)
8. [Seguridad](#)
9. [Optimizaciones de rendimiento](#)
10. [Instalación y puesta en marcha](#)
11. [Variables de entorno](#)
12. [Despliegue \(Render\)](#)
13. [Pruebas automatizadas](#)
14. [Estado conocido del sistema / deuda técnica](#)
15. [Historial de cambios relevantes](#)

1. Descripción general

Canje FTT (marca de producto: *FeelTheTickets* — *Canje*) es un sistema web para gestionar el canje físico de boletos de un evento: reemplaza la validación manual en puerta por un flujo digital con búsqueda de asistentes, canje individual y masivo, control de acceso por roles, auditoría completa de cada operación y actualizaciones en tiempo real entre puntos de venta.

El sistema es agnóstico al evento concreto: los boletos se cargan desde un archivo CSV al iniciar la operación ([backend/src/scripts/setup.js](#)), y las localidades/sectores del recinto se extraen dinámicamente de esos datos, no están escritas en el código.

2. Stack tecnológico

Backend

- Node.js + Express
- MongoDB Atlas + Mongoose
- JWT ([jsonwebtoken](#)) para autenticación
- [bcryptjs](#) para hash de contraseñas
- Winston para logging estructurado
- Helmet + [express-rate-limit](#) para seguridad HTTP
- Socket.IO para tiempo real
- Jest + Supertest para pruebas automatizadas

Frontend

- React 18 + Vite
- React Router (SPA)
- Bootstrap 5 + Bootstrap Icons + Font Awesome
- SweetAlert2 para confirmaciones/alertas
- Axios para llamadas a la API
- Socket.IO Client
- Context API para estado de autenticación

3. Estructura del proyecto

```

Canje FTT/
├── backend/
│   ├── src/
│   │   ├── app.js                # Punto de entrada: Express + Socket.IO +
│   │   └── middlewares globales
│   │       ├── config/
│   │       │   ├── database.js    # Conexión a MongoDB
│   │       │   ├── collections.js # Resolución de la colección de tickets activa
│   │       │   └── logger.js      # Winston (service: "canje-fft")
│   │       ├── models/           # Ticket, User, PuntoVenta, AuditLog
│   │       ├── controllers/      # Lógica de negocio por dominio
│   │       ├── routes/           # Definición de rutas Express
│   │       └── middleware/       # auth, authorize, auditLogger,
│   └── collectionSelector
│       ├── utils/                # validators, csvImporter, ecuadorTime, etc.
│       └── scripts/              # setup.js y utilidades de línea de comandos
│   ├── tests/unit/               # Suite de pruebas automatizadas (Jest)
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── pages/                # Login, Dashboard, TicketsPage, UsersPage,
│   │   │   ├── AuditPage, PuntosVenta, ChangePassword
│   │   │   └── components/       # Navigation, ProtectedRoute,
│   │   │   └── RoleBasedRedirect, Layout
│   │   ├── context/              # AuthContext
│   │   ├── services/             # api.js (Axios), socket.js (Socket.IO client)
│   │   ├── utils/                # validators (celular/nombre)
│   │   └── styles/               # theme.css

```

```
|   └─ package.json
|   └─ EVIDENCIAS_PASANTIA/          # Registro de actividades de pasantía y
documentación histórica
|   └─ render.yaml                  # Configuración de despliegue (backend +
frontend)
|   └─ Documentacion.md              # Este documento
```

4. Roles y permisos

Rol	Acceso
Jefe	Acceso total: dashboard, gestión de usuarios, puntos de venta, auditoría, canje individual y masivo, reimpresión.
Staff	Canje individual y masivo (habilitado a partir de julio 2026) restringido a su punto de trabajo, búsqueda de boletos. Sin acceso a administración.
Impresor	Contemplado en la interfaz y en varios controladores (reimpresión de boletos ya impresos), pero no está incluido en el enum de roles del modelo User (<code>backend/src/models/User.js</code> solo define <code>['jefe', 'staff']</code>). Ver sección 14.

El control de acceso se aplica en dos capas:

- **Frontend:** `ProtectedRoute` (con la prop `roles`) redirige a `/unauthorized` si el rol no corresponde; `Navigation.jsx` solo muestra los enlaces relevantes para el rol autenticado.
- **Backend:** middleware `authorize(...roles)` en cada ruta sensible, que devuelve `403` si el rol del usuario autenticado no está permitido.

5. Modelo de datos

User (colección **Usuarios**)

```
{
  nombre: String,           // requerido, solo letras (validado en el controlador)
  usuario: String,          // único, email, usado como login
  password: String,         // hash bcrypt (salt 12), nunca se serializa en las
respuestas
  rol: 'jefe' | 'staff',    // enum actual del schema (ver sección 14 sobre
'impresor')
  puntoTrabajo: String,     // requerido si rol === 'staff'
  primerAcceso: Boolean,    // fuerza cambio de contraseña en el primer login
  activo: Boolean,
  creadoPor: ObjectId       // referencia a User
}
```

Ticket (colección configurable, ver sección 14)

```

{
  'First Name': String,
  'Last Name': String,
  'Email': String,
  'Ticket': String,           // categoría/tipo de entrada
  'Seat': String,             // localidad – se usa para filtrar por punto de
venta
  'Transaction ID': String,
  'Ticket ID': String,        // único, indexado
  'Numero de Cedula': String,
  canjeado: Boolean,
  fechaCanje: Date,
  impreso: Boolean,
  fechaImpresion: Date,
  usuarioResponsable: ObjectId, // usuario que realizó el canje/impresión
(poblado en las respuestas)
  usuarioCanje: ObjectId,      // mismo usuario que usuarioResponsable; no se
popula en las consultas actuales
  puntoTrabajo: String,
  puntoCanje: String,
  quienRetira: 'Titular' | 'Titular Compra' | 'Otro',
  parentesco: String,          // solo si quienRetira === 'Otro'
  quienOtro: String,           // solo si quienRetira === 'Otro', solo letras
  celular: String,             // solo dígitos, 7-15 caracteres
  reimpressiones: [{ fecha, motivo, usuario, puntoTrabajo }]
}

```

PuntoVenta (colección PuntosVenta)

```

{
  nombre: String,              // único
  descripcion: String,
  localidades: [String],       // sin enum fijo – se extraen dinámicamente del CSV
importado
  activo: Boolean,
  creadoPor: ObjectId
}

```

AuditLog (colección AuditLogs)

```

{
  tipo: 'impresion' | 'reimpresion' | 'canje' | 'canje_masivo' | 'login' |
'logout' | 'creacion_usuario' | 'cambio_password',
  usuario: ObjectId,
  ticketId: String,
  transactionId: String,
  puntoTrabajo: String,
  detalles: Mixed,
}

```

```
ip: String,  
userAgent: String  
}
```

Solo expone endpoints de **lectura** (no hay edición ni borrado de logs desde la API), y un fallo al registrar auditoría nunca bloquea la operación principal que la originó.

6. API — Endpoints

Todas las rutas (excepto `/health`, `/api/health` y `POST /api/auth/login`) requieren un token JWT en el header `Authorization: Bearer <token>`.

Autenticación (`/api/auth`)

Método	Ruta	Acceso
POST	<code>/login</code>	Público
POST	<code>/change-password</code>	Autenticado
POST	<code>/logout</code>	Autenticado
GET	<code>/profile</code>	Autenticado

Usuarios (`/api/users`) — solo Jefe

Método	Ruta
GET	<code>/</code>
POST	<code>/</code>
PUT	<code>/:id</code>
DELETE	<code>/:id</code> (baja lógica)

Tickets (`/api/tickets`)

Método	Ruta	Acceso
GET	<code>/</code>	Autenticado (filtrado automático por punto de trabajo si el rol es Staff)
GET	<code>/stats</code>	Jefe
GET	<code>/transaction/:transactionId</code>	Jefe
POST	<code>/bulk-canje</code>	Jefe y Staff
POST	<code>/:id/print</code>	Jefe y Staff
POST	<code>/:id/reprint</code>	Jefe
POST	<code>/:id/canje</code>	Jefe y Staff

Puntos de venta (/api/puntos-venta)

Método	Ruta	Acceso
GET	/	Autenticado
POST	/	Jefe
PUT / DELETE	/:id	Jefe
GET	/staff/tickets	Staff/Impresor (forzado a su propio punto de trabajo)
GET	/:id/tickets, /:id/estadisticas	Autenticado — sin verificación de pertenencia al punto de trabajo del solicitante , ver sección 14
GET	/localidades/disponibles	Autenticado

Auditoría (/api/audit) — solo Jefe

Método	Ruta
GET	/ (filtros: tipo, usuario, ticketId, fechaInicio, fechaFin)
GET	/summary (agregados por tipo, por usuario y por día)

Salud

Método	Ruta
GET	/health
GET	/api/health

7. Actualizaciones en tiempo real (Socket.IO)

El sistema usa **Socket.IO** (no polling) para que los cambios de un ticket se vean instantáneamente en las pantallas de otros usuarios conectados, sin recargar.

- **Salas por contexto:** cada cliente se une a `punto-venta-{id}` (Jefe, según el punto de venta seleccionado) o `staff-{puntoTrabajo}` (Staff), de forma que un usuario solo recibe actualizaciones de su propio contexto.
- **Evento emitido:** `ticket-updated`, disparado desde el backend tras un canje o una impresión, con `{ action, ticket, timestamp }`.
- **Autenticación:** el token JWT se envía en el *handshake* de conexión.
- **Reconexión automática:** hasta 5 intentos con backoff, con *fallback* a polling HTTP si el WebSocket no está disponible.
- **Pausa inteligente en el frontend:** las actualizaciones entrantes se posponen mientras el usuario está escribiendo en un buscador o interactuando con un formulario, para no interrumpirlo a mitad de una acción.

8. Seguridad

Mecanismo	Implementación
Autenticación	JWT firmado con <code>JWT_SECRET</code> , expiración de 8 horas
Contraseñas	<code>bcryptjs</code> , salt de 12 rondas; nunca se devuelven en las respuestas (<code>toJSON()</code> las elimina)
Autorización por rol	Middleware <code>authorize(...roles)</code> en cada ruta sensible del backend
Cabeceras HTTP	Helmet (protección XSS, sniffing MIME, clickjacking)
CORS	Lista blanca explícita (<code>localhost</code> en desarrollo; dominios configurados + <code>*.onrender.com</code> en producción)
Rate limiting	200 peticiones/minuto por IP; excluye <code>/health</code> y rutas <code>check-changes</code>
Auditoría	Toda operación sensible (login, logout, cambio de contraseña, creación de usuario, canje, canje masivo, impresión, reimpresión) genera un único registro en <code>AuditLog</code> , con IP capturada de forma síncrona antes de responder
Validación de formato	Celular: solo dígitos (7–15); nombres: solo letras — aplicada en frontend (filtrado en vivo) y backend (<code>backend/src/utils/validators.js</code>), como defensa en profundidad

Ver también la sección 14 para las brechas de seguridad conocidas y no corregidas (verificación de pertenencia de punto de trabajo, condición de carrera en el canje).

9. Optimizaciones de rendimiento

- **Índices de MongoDB** en los campos de consulta frecuente: `Ticket ID` (único), `Seat`, `Transaction ID`, `Numero de Cedula:`, `updatedAt`, y compuestos (`Ticket + updatedAt`, `canjeado + puntoTrabajo`, `impreso + puntoTrabajo`).
- `.lean()` en consultas de solo lectura, para reducir uso de memoria.
- `maxTimeMS()` en las consultas críticas, para evitar que una query cuelgue el servidor indefinidamente.
- **Consultas en paralelo** (`Promise.all`) donde no hay dependencia entre ellas (p. ej. listado + conteo total para paginación).
- `bulkWrite` para el canje masivo, en vez de actualizaciones una por una.
- **Pool de conexiones de MongoDB** ampliado, con reintento de lecturas habilitado.

10. Instalación y puesta en marcha

Requisitos

- Node.js 16+
- Cuenta de MongoDB Atlas (o instancia propia de MongoDB)
- Un archivo CSV con los datos de boletos del evento

Pasos

```
# 1. Instalar dependencias
cd backend && npm install && cd ..
cd frontend && npm install && cd ..

# 2. Configurar variables de entorno del backend
cp backend/.env.example backend/.env
# Editar backend/.env: MONGODB_URI, JWT_SECRET, DEFAULT_PASSWORD

# 3. Importar el CSV del evento y crear el acceso inicial
cd backend
node src/scripts/setup.js <ruta-al-csv-del-evento>
cd ..

# 4. Iniciar los servidores
# Terminal 1
cd backend && npm run dev
# Terminal 2
cd frontend && npm run dev
```

`setup.js` es el **único** procedimiento soportado para crear el primer usuario administrador: importa los boletos, extrae las localidades desde la columna `Seat`, crea/actualiza el Punto de Venta correspondiente, y crea el usuario `sistema` (rol Jefe, contraseña `sistema-inicial`) si todavía no existe ningún usuario con rol Jefe en la base de datos. El servidor (`config/database.js`) **no** crea ningún usuario automáticamente al arrancar.

Login inicial: usuario `sistema` / contraseña `sistema-inicial` — debe cambiarse en el primer acceso (el sistema lo exige).

Scripts disponibles

Backend

```
npm run dev      # nodemon
npm start        # producción
npm test         # suite de pruebas automatizadas (Jest)
```

Frontend

```
npm run dev      # servidor de desarrollo (Vite)
npm run build     # build de producción
npm run preview   # preview del build
npm run lint      # ESLint
```

11. Variables de entorno

Backend (`backend/.env`)


```
NODE_ENV=development
PORT=5002
MONGODB_URI=mongodb+srv://usuario:password@cluster.mongodb.net/tu-base
COLLECTION_NAME=nombre-de-la-coleccion-de-tickets
JWT_SECRET=un-secreto-largo-y-aleatorio
DEFAULT_PASSWORD=contraseña-inicial-para-nuevos-usuarios-staff
LOG_LEVEL=info
CORS_ORIGIN=https://tu-frontend-en-produccion.com
```

Frontend (frontend/.env.local)

```
VITE_API_URL=http://localhost:5002
```

No debe versionarse ningún valor real de `MONGODB_URI` ni de `JWT_SECRET` — en despliegue (Render), estas variables se configuran manualmente desde el panel del servicio (`render.yaml` usa `sync: false` para `MONGODB_URI`).

12. Despliegue (Render)

El proyecto se despliega como dos servicios independientes (ver `render.yaml`):

- **Backend:** servicio web Node (`rootDir: backend`), build `npm install`, start `npm start`.
- **Frontend:** sitio estático (`rootDir: frontend`), build `npm install && npm run build`, publica `dist/`, con reglas de `rewrite: /api/*` hacia el backend, `/*` hacia `index.html` (necesario para que las rutas de React Router funcionen al recargar).

Antes de anunciar el sistema como disponible, debe ejecutarse `node src/scripts/setup.js <csv-del-evento>` contra la base de datos de producción, para crear el punto de venta y el acceso inicial.

Verificación posterior al despliegue: `GET /health` y `GET /api/health` en la URL pública del backend; login con el usuario `sistema`; confirmar en la consola del navegador que Socket.IO conecta.

13. Pruebas automatizadas

El backend cuenta con una suite de pruebas unitarias en `backend/tests/unit/`, ejecutable con `npm test` (Jest), que **no requiere conexión a una base de datos real** — los modelos de Mongoose se mockean.

Archivo	Cobertura
<code>validators.test.js</code>	Validación de formato de celular y nombre
<code>authMiddleware.test.js</code>	Middleware de autenticación y autorización por rol
<code>routesAuthorization.test.js</code>	Cableado de rutas: ausencia de auditoría duplicada, autorización real de <code>/tickets/bulk-canje</code> por rol
<code>ticketCanjeValidation.test.js</code>	Validación de canje individual y masivo antes de tocar la base de datos

Archivo

Cobertura

`userValidation.test.js`

Validación de nombre en creación/edición de usuarios

No existe (todavía) una suite de pruebas automatizadas para el frontend.

14. Estado conocido del sistema / deuda técnica

Estos puntos están identificados y documentados deliberadamente, no son errores desconocidos:

#	Descripción	Impacto
1	El modelo <code>User</code> no incluye <code>'impresor'</code> en el enum de <code>rol</code> , aunque el resto del sistema (frontend, varios controladores) sí lo contempla. Crear un usuario con ese rol es rechazado por la validación de Mongoose.	Bloquea la creación de usuarios Impresor hasta que se decida agregar el rol al enum o retirar las referencias restantes.
2	<code>GET /api/puntos-venta/:id/tickets, /:id/tickets/check-changes</code> y <code>/:id/estadisticas</code> no verifican que el <code>:id</code> de la URL corresponda al punto de trabajo del usuario autenticado. La interfaz actual no explota esto (Staff usa <code>/staff/tickets</code> , que sí está protegida), pero a nivel de API un usuario Staff/Impresor podría solicitar datos de un punto de venta que no es el suyo.	Riesgo de exposición de datos entre puntos de venta si se accede directamente a la API.
3	El canje individual (<code>canjeTicket</code>) y masivo (<code>bulkCanjeTickets</code>) validan <code>canjeado === false</code> y luego guardan en pasos separados, no de forma atómica. Dos peticiones casi simultáneas para el mismo ticket podrían generar doble procesamiento.	Baja probabilidad en operación normal; se mitigaría con <code>findOneAndUpdate</code> condicional en vez de <code>find + save</code> .
4	El frontend muestra el botón de reimpresión al rol Impresor, pero el backend solo autoriza <code>jefe</code> en <code>POST /tickets/:id/reprint</code> .	Un usuario Impresor recibiría <code>403</code> al intentar reimprimir desde la interfaz.
5	<code>backend/src/models/Ticket.js</code> fija la colección de Mongoose como <code>'FechaUno'</code> en la definición del modelo base, mientras que <code>config/collections.js</code> resuelve la colección activa (<code>req.TicketModel</code>) a partir de <code>process.env.COLLECTION_NAME</code> (por defecto <code>'Lumineers_Canje'</code>). Los controladores que usan <code>req.TicketModel</code> y los que importan <code>Ticket</code> directamente pueden terminar apuntando a colecciones distintas según el valor real de esa variable de entorno.	Requiere confirmar/unificar cuál es la colección real en uso antes de un nuevo evento, para evitar leer o escribir en la colección equivocada.
6	<code>src/utils/csvImporter.js</code> (usado solo por <code>src/scripts/importCSV.js</code> y <code>scripts/init.js</code> , no por el flujo oficial <code>setup.js</code>) mapea las columnas del CSV a nombres de campo que no coinciden con el schema real de <code>Ticket</code> .	Código muerto con un bug latente; no afecta la importación oficial, pero debería eliminarse o

#	Descripción	Impacto
		corregirse para evitar que alguien lo use por error.
7	Credenciales de MongoDB expuestas en el historial de Git de versiones anteriores de <code>render.yaml</code> (el archivo actual ya no las contiene en texto plano).	Pendiente de acción externa: rotar la contraseña del usuario de base de datos desde el panel de MongoDB Atlas.
8	Existen archivos de código sin uso activo (<code>App_backup.jsx</code> , <code>App_render.jsx</code> , <code>Navigation_fixed.jsx</code> , <code>TicketsPage_fixed.jsx</code> , <code>src/scripts/createAdmin.js</code> , entre otros) que no forman parte del flujo actual de la aplicación.	Sin impacto funcional; candidatos a limpieza para reducir confusión futura.

15. Historial de cambios relevantes

- **Base de arquitectura:** separación en 3 capas (frontend SPA, API REST, MongoDB), con localidades extraídas dinámicamente del CSV en vez de una lista fija en el código, y eliminación de la lógica de múltiples colecciones/cronograma por fecha que existía en una versión anterior del proyecto.
- **Tiempo real:** reemplazo de polling por Socket.IO, con salas por punto de venta/punto de trabajo y pausa inteligente de actualizaciones mientras el usuario interactúa con la interfaz.
- **Rendimiento:** índices adicionales, timeouts en consultas, `.lean()`, consultas en paralelo y `bulkWrite` para el canje masivo.
- **Canje masivo para Staff:** originalmente restringido solo a Jefe; habilitado también para Staff a partir de julio de 2026, junto con validaciones de formato de celular y nombre en frontend y backend.
- **Corrección de auditoría duplicada:** se eliminó el registro doble de `AuditLog` (middleware + controlador) en canje, canje masivo, cambio de contraseña y logout, y se corrigió la captura de IP para que ocurra antes de enviar la respuesta.
- **Corrección de detección de usuario en el canje:** el frontend leía un campo del ticket que el backend nunca poblab; se corrigió para leer el campo correcto.
- **Corrección de contraste en la tabla de Tickets:** una regla CSS propia tenía más especificidad que la clase de Bootstrap usada para la cabecera oscura, dejando el texto ilegible; se corrigió la regla.
- **Suite de pruebas automatizadas:** incorporación de pruebas unitarias con Jest/Supertest para autenticación, autorización y validaciones de formato, donde antes solo existía verificación manual o por inspección de código.